



Software Engineering Department

Braude College of Engineering

Final Project – Phase B (61999)

Automated System for University Exam Scheduling

Project Code: 26-1-D-12

By:

Alon Dahan

Karyn Arama

Advisor:

Dr. Julia Sheidin

Dr. Avital Shulner

<https://github.com/KarynA19/Exam-Optimizer.git>

Table of Contents

1. Problem Review and Background	3
1.1 Problem Description	3
1.2 General Background	3
1.3 Literature Review and Theoretical Framework	4
2. General Description	4
3. Solution Description and System Architecture	5
3.1 How the Two Sides Work Together	6
3.2 How Data Is Described	6
3.3 How Constraints Are Checked	7
3.4 How the Scheduler Works: CP-SAT Constraint Programming	8
3.5 System Performance and Evaluation Metrics	9
3.6 Manual Move Logic	10
4. Development Process	10
4.1 Understanding the Context	10
4.2 Specifying Requirements	10
4.3 Designing the Solution	11
4.4 Testing and Iteration	11
4.5 Stakeholder Contact During Development	11
5. Challenges and Solutions	12
6. Results and Conclusions	13
6.1 Project Outcomes	13
6.2 Project Metrics Evaluation	13
6.3 User Feedback	14
7. Lessons Learned	14
7.1 Development Approach	14
7.2 Team Collaboration	15
7.3 What We Would Do Differently	15
8. Application Screenshots	16
8.1 Initial Setup Interface – Defining Exam Period and Spacing Rules	16
8.2 Excluded Dates Interface – Blocking Unavailable Exam Periods	17
8.3 Fixed Exams Interface – Locking Exams to Required Dates	17
8.4 Courses Interface – Managing Course Data for Scheduling	18
8.5 Schedule Interface – Viewing Generated Exam Timetable Options	19
9. References	20
10. User Guide	21
10.1 Introduction	21
10.2 Getting Started	21
10.3 Nominal Workflow Overview	21
10.4 Setup – Initial Configuration	22
Step 1: Initial Setup	22
Step 2: Excluded Dates	22
Step 3: Fixed Exams	22

Step 4: Courses	23
Step 5: Generate Schedule Options	23
10.5 Schedule Workspace – Reviewing Timetables	24
10.6 Manual Exam Adjustments	25
10.7 Continuing Your Work	26
11. Maintenance Guide	26
11.1 Technology Stack	26
11.2 Development Environment Setup	27
11.2.1 Prerequisites	27
11.2.2 Installing Dependencies	27
11.2.3 Environment Variables	28
11.2.4 Running the Development Server	29
11.2.5 Project Structure	30
11.3 Backend Architecture	32
11.3.1 API Routes Structure	32
11.3.2 Core API Endpoints	32
11.3.3 Request Bodies	34
11.3.4 Service Layer	34
11.4 Frontend Architecture	35
11.4.1 Vite + React SPA	35
11.4.2 Application Routes	36
11.4.3 Component Organization	36
11.4.4 State Management	37
11.5 Data Persistence Architecture	38
11.5.1 Data Flow	38
11.5.2 MongoDB Persistence	39
11.6 Development Workflows	40
11.6.1 Git Workflow	40
11.6.2 Backend Workflow	40
11.6.3 Frontend Workflow	41
11.7 Debugging and Troubleshooting	41
11.7.1 HTTP Status Codes Reference	41
11.7.2 Common Issues and Solutions	42
11.8 Exam Optimizer – Specific Features	43
11.8.1 Scheduling Policies Encoded in Code	43
11.9 Future Maintenance	45

1. Problem Review and Background

1.1 Problem Description

The final examination scheduling process in academic departments is an extremely complex task, demanding simultaneous compliance with a wide array of conflicting constraints stemming from faculty availability, religious and academic calendar restrictions, classroom capacity, and mandatory institutional rules on student examination load. Manually managing this process consumes valuable time resources, increases the risk of scheduling errors (such as overlaps or violations of holy day restrictions), and often results in sub-optimal timetables for both lecturers and students.

This project presents the development of a Computerized Examination Scheduling System designed to automate this process for the Administrative Assistant to the Head of the Software Engineering Department, while ensuring full adherence to the college's policies and the department's specific requirements. The system utilizes optimization algorithms to generate a comprehensive schedule by implementing a multi-layered constraint model.

The model includes specific consideration for the following factors: distinction between regular faculty members and external lecturers (where only faculty constraints are considered); precise calendar restrictions (holidays, fast days, and fixed dates for Moed A and Moed B periods); mandatory student load rules (up to 3 weekly exams per student); handling of cross-departmental exams (held simultaneously); lecturer constraints (no two exams for the same lecturer in the same week).

The final system offers the user initial automated scheduling and the capability for a Re-shuffle based on secondary parameters, providing a flexible, accurate, and immediate solution to the scheduling problem.

1.2 General Background

Every semester, the Administrative Assistant (Ora) at the Software Engineering Department sits down with a spreadsheet and attempts the near-impossible: building a final exam timetable that works for everyone. Courses cannot overlap for students on the recommended track. Prerequisite pairs need breathing room between them. High-failure-rate courses demand extra spacing. Religious holidays, institutional blocked days, and pre-agreed fixed exam slots all carve away at an already limited window of available dates. And once a draft is done, a single change to a lecturer's conflict, a late-added fixed date, can cascade into a dozen new violations that need to be untangled by hand.

This is not a scheduling problem in the casual sense. It is a multi-constraint combinatorial optimization problem that becomes harder with each course added to the curriculum. For a large department managing two separate tracks (Software Engineering and Information Systems) across eight semesters, the manual approach is not just inefficient; it is a recurring

source of errors that go undetected until the timetable is already published. Exam Optimizer was built to change that.

1.3 Literature Review and Theoretical Framework

University examination timetabling is an NP-hard combinatorial problem: exams must be assigned to a limited set of dates under institutional rules, and manual scheduling does not scale well for large departments (Burke & Petrovic, 2002). Our work follows a curriculum-based view. Conflicts are defined by the recommended academic track (semester and prerequisites) rather than by individual students' enrollment, thereby reducing complexity and aligning with the way the Software Engineering Department plans Moed A (Bonutti et al., 2012).

In the literature, the problem is modeled as a Constraint Satisfaction Problem (CSP), with hard constraints that define feasibility (e.g., blocked holidays, minimum gaps, fixed dates) and soft constraints that define quality (e.g., wider spacing for difficult courses) (Barták et al., 2010; Burke & Petrovic, 2002). Studies using optimization and decision-support systems report similar rule sets (Aslan & Kapanoglu, 2017; Güler et al., 2021). Common solution approaches include fast heuristics (often weak on quality), meta-heuristics such as genetic algorithms (flexible but no feasibility guarantee) (Pillay & Banzhaf, 2010; Elen, 2022), and constraint programming, which expresses logical rules directly and fits medium-sized instances when hard constraints must hold (Barták et al., 2010).

Phase A compared these options and selected constraint programming with Google OR-Tools CP-SAT. Exam Optimizer implements that choice: allowed dates form the domain, gap and fixed-exam rules are hard constraints, and solutions are ranked by a score. This condensed review supports the Phase B design; the full comparative discussion and figures appear in Phase A (Capstone Project Phase A, 26-1-D-12).

2. General Description

Exam Optimizer is a web-based decision-support tool for final exam scheduling in the Software/Information Systems Engineering Department. It does not attempt to replace the administrative assistant's judgment; it gives that judgment something solid to work with.

The system emerged from a concrete frustration: Ora (assistant to the Head of Department) was spending significant time each semester manually building exam timetables, with no systematic way to check for constraint violations or explore alternative arrangements. A change to one exam could break others in ways that were not immediately visible. The process was opaque, slow, and dependent on institutional memory rather than documented rules.

The deliverable addresses this directly. The application walks the coordinator through a structured setup: define the Moed A/B windows and their gap rules, block out holidays and unavailable dates, lock in any pre-agreed fixed exams, and import or enter the course list. From there, the solver generates up to five candidate timetables simultaneously, each satisfying every hard constraint, each ranked by a spacing quality score. The coordinator can compare them

side by side, inspect the dependency graph to understand why a course is hard to move, and apply manual adjustments through a guided calendar interface that shows in real time whether a proposed move is valid, and what it costs in score.

The system is organized as a standard client–server product. The frontend is a React single-page application that handles all data entry, visualization, and interaction. The backend is a Python FastAPI service that hosts the scheduling logic and exposes a REST API covering four core operations: validate, solve, manual-move, and explain-move. The algorithmic core runs Google OR-Tools CP-SAT. A dedicated validation layer checks inputs and proposed timetables both before and after solving, surfacing issues with enough context course code, date, constraint type for the coordinator to act on them immediately.

Working drafts are persisted in MongoDB, so the coordinator can close the tab and return to a session without losing progress. Saved initial setups (Moed A/B windows, gap rules, constraint weights) can be reused across years, reducing repeated configuration effort.

3. Solution Description and System Architecture

Exam Optimizer is built as a two-tier web application: a browser-based frontend that the coordinator interacts with, and a backend server that handles all the scheduling intelligence (see Fig.1) . These two sides communicate over the internet using a standard request–response format (HTTP/JSON).

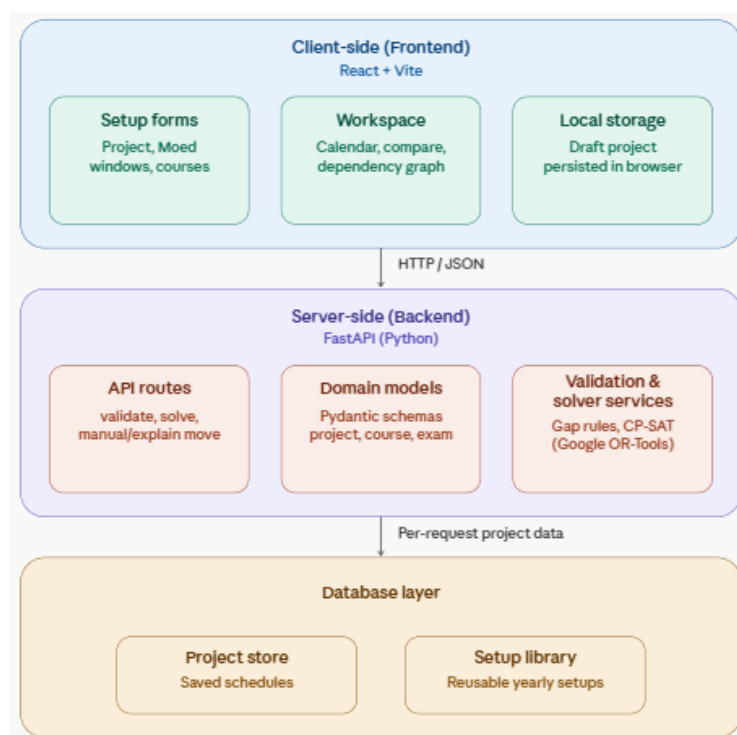


Fig 1: System Architecture

How the two sides work together

The structural organization and responsibilities of the system's frontend and backend components are outlined in Fig 2. The frontend is built with React. Everything the coordinator sees setup forms, the calendar, the comparison table. When the coordinator makes a change, the frontend sends the full current project (courses, windows, constraints, and any existing solutions) to the backend with each request. The backend stores nothing between requests: it receives everything it needs, computes an answer, and returns it. The project draft is saved in MongoDB, so the coordinator can close the tab and return later without losing work.

Package / module	Responsibility
frontend — App, forms	Capture project setup; trigger validate/solve/import
frontend — workspace	Visualize solutions; manual move UX
frontend — storage	Persist draft project in browser
backend — routes	Map HTTP operations to services
backend — models	Data structures and validation schema
backend — validation	Allowed dates, constraints, scoring, move checks
backend — solver	CP-SAT model, multiple solutions
backend — course_import	Excel → course list

Fig 2: Package structure

The backend is built with FastAPI (Python). It exposes five operations the frontend calls:

- **Validate:** checks the project for setup problems (e.g., a fixed exam on a blocked date)
- **Solve:** runs the scheduling algorithm and returns up to five candidate timetables.
- **Manual Move:** checks whether moving one exam to a proposed date is valid and, if so, returns the updated solution with a new score.
- **Explain Move:** the same check, called once per calendar day when an exam is selected, so each day can be colored green, yellow, or red.
- **Import courses:** parses an Excel (.xlsx) file and returns a structured list of courses.

How data is described

Before scheduling logic runs, the system must agree on what constitutes a "project". On the backend, this is handled by Pydantic models: every course, Moed window, scheduled exam, and validation issue has a defined shape. Incoming requests are validated before any algorithm runs. The frontend mirrors the same structure in TypeScript types so both sides speak the same language.

How constraints are checked

The validation service is the system's rule-checker. For any proposed timetable it:

- Builds allowed exam dates from each Moed window, removing Saturdays and excluded ranges (holidays, blocked periods).
- Checks every pair of courses (not just neighbors) for applicable gap rules (e.g., same semester, same department → same-semester gap (default: 2 days))
- Enforces fixed exams on their exact dates, no exams on Saturdays or excluded days, and every exam inside its Moed window.
- Allows at most one Friday exam per department (SW or IS or Both) per Moed window.

The same logic runs before solving (to catch configuration errors), after solving (to confirm each returned timetable), and on every manual move, so the UI always reflects the real rules. (see Fig.3)

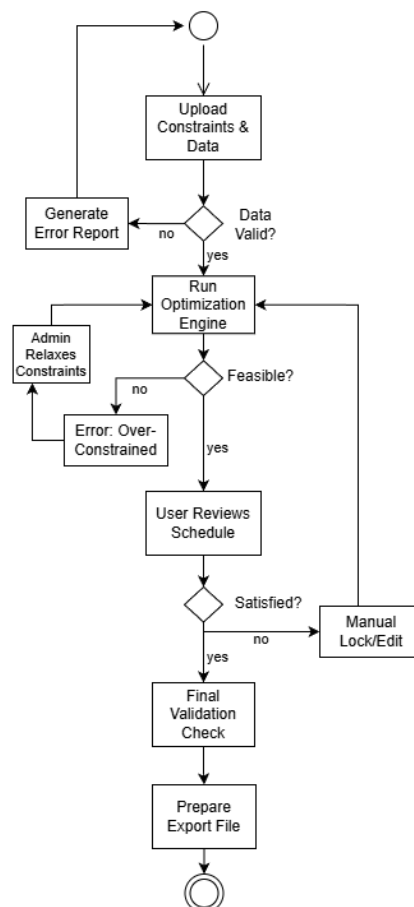


Figure 3: Flow chart diagram

How the scheduler works: CP-SAT constraint programming

The core engine is Google OR-Tools CP-SAT. Phase A compared heuristics, meta-heuristics, and constraint programming; we chose CP-SAT because departmental rules are naturally expressed as logical constraints and feasibility must be guaranteed, not approximate. The scheduling problem was modeled in CP-SAT using several design decisions, summarized in Table 1.

Table 1: CP-SAT Modeling Decisions and Trade-offs

Design choice	What we did	Tradeoff
Allowed dates	Saturdays and excluded ranges are removed before modeling; only remaining dates enter the domain.	Pre-filtering shrinks the search space; blocked days never appear as candidates.
Friday limit	For each course, a Boolean "is on Friday" flag is linked to the date variable via allowed assignments; per department, $\text{sum}(\text{Friday flags}) \leq 1$.	Reifies day-of-week logic within CP-SAT rather than post-processing to guarantee feasibility.
Even distribution	The solver tracks cumulative exams scheduled by date and penalizes deviation from a uniform target spread (weighted by <code>global_spacing_weight</code> , default 4).	Rewards timetables that "breathe" across the window, not only pairwise gaps.
Multiple solutions	After each solve (up to 10 seconds per iteration), we add a constraint that forces at least one course to change its date, then search for up to 5 distinct timetables per Moed window.	The diversity constraint is simple and effective; it does not guarantee maximum pairwise difference but produces genuinely different options for comparison.
Moed A / Moed B	Each Moed window is solved independently, then results are combined into full timetables.	Scales well and matches how windows are configured; cross-Moed gap rules are not needed because gap checks apply within the same window.

Post-solve validation	Every solver output is re-checked by the validation service and re-scored by <code>score_solution()</code> .	Duplicates some logic already in the objective, but ensures the API never returns a timetable that violates a rule; the solver's objective and the coordinator-facing score remain consistent.
------------------------------	--	--

The implemented system was evaluated in terms of runtime, solution diversity, and schedule quality. The main results are summarized in Table 2.

Table 2: System Performance and Evaluation Metrics

Metric	Value	How measured
Solver time limit	10 seconds per solution iteration	<code>CpSolver.parameters.max_time_in_seconds = 10</code>
Solutions returned	Up to 5 per solve request	<code>max_solutions</code> : 5 from frontend; diversity constraint between rounds
End-to-end solve time	Usually under 15 seconds for feasible ~50-course instances; worst case bounded by ~50 s per Moed window (5×10 s)	Manual timing during development; within Phase A's 60-second target
Infeasible input	Solver returns 0 solutions with a clear error (e.g., <code>no_feasible_schedule</code>)	Tested with tight windows and conflicting gap rules
Quality comparison	Solutions ranked by score and spacing deviation; coordinator compares up to 5 options side by side	Comparison dashboard

How the coordinator knows a solution is "good":

1. It passes all hard rules (no red issues).
2. It has a higher score and lower spacing deviation than alternatives.

3. Manual moves that improve local placement are immediately validated; the calendar shows green/yellow/red before any change is applied.

How manual moves work

When the coordinator selects an exam, the frontend asks the backend to evaluate every schedulable day in the window (one request per day, run in parallel). The backend runs full validation and scoring for each hypothetical placement. The calendar colors each day:

- Green — valid, and score stays equal or improves;
- Yellow — valid, but score drops;
- Red — hard constraint violated.

Clicking Apply move sends one confirmation request; the updated solution is saved locally.

4. Development Process

The development of Exam Optimizer followed the structured approach defined in Phase A: a user-centered design (UCD) process that moves from understanding the scheduling context, through requirements and design, to implementation and evaluation [Capstone Project Phase A]. Phase A established the problem domain, the literature-based choice of constraint programming, functional and non-functional requirements, and low-fidelity interface concepts for the administrative assistant. Phase B focused on turning that foundation into a working, testable system while keeping continuous alignment with the real workflow described by the department's scheduling coordinator.

4.1 Understanding the context

In Phase A, we analyzed how final exams are scheduled today: interviews and observations with Ora for timetables, an examination of the existing Excel-based process (calendar exclusions, spacing for difficult courses, fixed dates, and manual entry into college systems). That material became the baseline for Phase B: every major feature in the delivered software, like blocked dates, course list, fixed exams, automated solve, and manual adjustment, maps to a point in this phase.

4.2 Specifying requirements

The requirements from Phase A [10] were refined into concrete backlog items for implementation. Because the full vision includes faculty rules, student weekly load limits, cross-department synchronization, and Moed B, we explicitly prioritized what would appear in the first deliverable: Moed A date assignment for department courses under a curriculum-based constraint model (semester gaps, prerequisite-related gaps, high-failure spacing), with configuration of gap lengths and support for multiple solver solutions. Requirements not

implemented in code were retained in the project book as future scope, so the documentation remains honest with respect to both Phase A and the repository.

4.3 Designing the solution

Before coding, we translated requirements into a high-level architecture, a shared data model, and an operational flow. Phase B realized these ideas in React components and FastAPI endpoints. The algorithm design stayed close to the Phase A decision to use Google OR-Tools CP-SAT, but the implementation adopted a single CP-SAT pipeline with post-solve validation and scoring, rather than the two-stage “heuristic then optimizer” flow described in the early documents; simpler to implement and sufficient for the department’s course scale.

4.4 Testing and iteration

Throughout implementation, the testing process was highly iterative, as the system did not work perfectly on the first run and required a dedicated refinement phase. We used automated unit tests for the solver and course import (`test_solver.py`, `test_course_import.py`) to verify feasibility and parsing behavior, as well as repeated manual end-to-end scenarios (empty window, holidays only, infeasible constraints, fixed exam on an excluded day, multiple solutions, invalid move). Initial rounds of these tests revealed critical edge cases and configuration conflicts; these issues were directly fed back into validation messages and UI flows. This iterative approach allowed us to systematically fix bugs and refine the application, ensuring it fully aligns with Phase A’s testing plan (data import, hard constraints, soft quality, performance) and is applied to the subset of rules present in the build.

4.5 Stakeholder contact during development

Throughout the development process, we maintained regular communication with stakeholders to gather feedback and validate our implementation. Periodic meetings with our supervisor, Dr. Julia Sheidin, who also serves as a project coordinator, provided continuous guidance and helped prioritize features based on real-world needs. Crucially, we held feedback sessions with Ora, the department’s administrative assistant, to align the system with her manual workflow. Her practical insights directly shaped the UI/UX; she emphasized making button actions explicit, designing the calendar view to closely mirror the layout of her familiar Excel sheets, and ensuring language consistency rather than a mix of English and Hebrew. Furthermore, based on her input, we prioritized course codes over course names, as she primarily recognizes courses by their numbers.

5. Challenges and Solutions

During development, we faced practical problems on three levels: what to build, how to model scheduling in code, and how two developers could deliver a working demo on time.

Challenge 1: Formalizing department rules in CP-SAT

Ora's rules are expressed in calendar terms, not as mathematical variables. We had to decide what is mandatory versus preferred, and encode both in a CP-SAT model.

Solution: Each course gets a date variable over allowed days only; fixed exams are pinned to a single value. For every relevant course pair, we enforce $|\text{date}(A) - \text{date}(B)| \geq \text{minimum gap}$. Minimum gaps, Saturdays, excluded dates, and the Friday-per-department limit are hard constraints; even spreading and avoiding Fridays are optimization goals. Every solver output is re-validated before display.

Challenge 2: UI/UX Tailoring

The main issue arose when attempting to display the entire schedule that adheres to Ora's original Excel sheet and making every detail visible (multiple solutions, conflicts, real-time re-shuffling, exam locking).

Solution: We decided on displaying the calendar in a periodic manner, where the days and dates are the Y-axis and semesters are on the X-axis. Thus, every exam finds its coordinates of semester to date. As for the details, we used sticky-floating text and buttons. Text to display latest action and errors. Two buttons, one to open the conflict bubble to avoid information flooding and one for re-shuffling. Exams on the schedule are right-clickable with the options of locking/editing and can also be moved by simply dragging them around, when held they display red/orange/green boxes to signal on conflicts.

Challenge 3: Multiple distinct timetables

Ora needs options to compare, not a single answer; CP-SAT tends to repeat the same arrangement. When she clicks on an exam, each day must show whether a move is legal and how it affects the score.

Solution: After each solution, we forced at least one course to move, then searched again (up to 5 rounds, 10 s each). We also gave her the option to lock an exam to a specific date and then re-shuffling around that.

Challenge 4: Understanding the real user workflow

At first, we believed we understood Ora's main difficulties, but after showing her the system, we realized that some workflow details were missing. The challenge was not only solving the schedule, but also presenting it in a way that matched her Excel-based work habits.

Solution: We updated the UI according to Ora's feedback: making the calendar more similar to Excel, emphasizing course codes, using clearer action buttons, and keeping the language consistent. This helped make the system not only technically correct, but also easier and more natural for the real user to use.

6. Results and Conclusions

6.1 Project Outcomes

Exam Optimizer was developed as a working web application that addresses the main scheduling problem identified in Phase A: building an exam timetable under departmental rules, with less manual work in Excel and fewer constraint violations.

The delivered system supports the full nominal workflow: define the exam window and blocked dates, enter or import courses, lock fixed exams, run the optimizer, compare several solutions, and adjust dates with immediate validation feedback.

Achieved objectives:

- Scheduling with blocked dates and configurable minimum gaps between related courses
- CP-SAT optimization (Google OR-Tools) with multiple solution options
- Validation before and after solving, with clear issue messages
- Fixed exams respected by the solver
- Manual moves with validation (and explanation of conflicts)
- Excel import for course lists
- Web UI for setup and review (calendar, comparison, dependency view)

6.2 Project Metrics Evaluation

In Phase A, we set several success metrics. For Exam Optimizer, our assessment is as follows.

Hard constraints: 100% Achieved. Through automated testing (test_solver.py), 100% of the generated timetables were found to completely reject violations involving excluded dates, fixed

exams, and mandatory gaps. The system also successfully blocked 100% of invalid manual moves during testing.

Speed: A typical department-sized list (~50 courses) is solved within the 60-second target from Phase A; runs are usually much faster. Achieved.

Schedule quality: Fully Achieved. The metric for ranking schedules was fully implemented via a data-driven scoring module (`score_solution()`) based on spacing weights and distribution deviation. Visual quality confirmation was achieved through dynamic color indicators (Green/Yellow/Red) on the calendar.

Re-shuffle: Fully Achieved. The manual re-shuffle workflow was fully realized, enabling real-time validation checks and immediate score recalculations for any manual move. Full automatic multi-course chain rescheduling was deferred to future expansions.

6.3 User Feedback

The system was evaluated in two ways: alignment with Phase A stakeholder input (interviews and requirements with the department's administrative assistant) and hands-on testing of the final product.

During Phase A, we validated needs and workflow with the scheduling coordinator. In Phase B, we used that input to keep the interface step-based and calendar-oriented, suitable for a non-technical user.

Administrator feedback:

“The calendar view makes it easier to see the whole exam period at once.”

“Getting several schedule options is useful before fixing individual exams.”

“Looks good.”

7. Lessons Learned

Developing Exam Optimizer taught us lessons about scope, teamwork, and the importance of focusing on the real user workflow.

7.1 Development Approach

- **Start with a clear MVP.** Phase A described a large system. Trying to build everything at once would have risked the product. Defining exam + core constraints first was the most important decision.

- **Build the engine before polishing the UI.** Validation and the solver had to work early. Once the API returned real solutions, the frontend had a stable target.
- **Question every “extra” feature.** Tools that do not help the administrator in the demo (e.g., heavy deployment setup or features from other projects) should be deferred. Time is better spent on clear errors and a reliable solve button.

7.2 Team Collaboration

- **Share one data contract.** Backend (ScheduleProject in Pydantic) and frontend (TypeScript types) must be in sync. Early agreement on JSON structure reduced integration bugs.
- **Integrate often.** One developer on the backend and one on the frontend work only if you merge and test the full flow regularly, not only in the last week.
- **Communicate blockers.** Short updates on what works (solve, import) and what does not (infeasible model) kept both sides aligned.

7.3 What We Would Do Differently

- **Integrate early and often:** Merge code frequently to catch integration issues before they compound.
- **Prioritize ruthlessly:** Focus on features that directly serve users, not technical “nice-to-haves.”
- **Plan for refactoring:** Allocate time in the schedule for code cleanup and technical debt management.

8. Application Screenshots

8.1 Initial Setup Interface – Defining Exam Period and Spacing Rules

The screenshot displays the 'Initial Setup' interface of the Exam Optimizer system. The interface is divided into several sections:

- Left Sidebar:** Contains navigation links: 'Setup' (selected), 'Initial Setup', 'Excluded Dates', 'Fixed Exams', 'Courses', and 'Schedule'.
- Top Bar:** Includes 'Refresh Saved Setups' and 'Logged in: orad'.
- ACTIVE WORKSPACE:** Shows the current workspace as 'Spring Moed A'.
- Initial Setup Section:** Contains a 'Project Name' field with 'Spring Moed A', a 'Moed Count' field with '1', and a 'New Setup' button. Below this is a 'Moed A' section with 'Start Date' (15/06/2026), 'End Date' (15/07/2026), and 'GAPS (DAYS)' for 'Same Sem' (3), 'Prereq' (3), and 'High Fail' (3).
- Current Entry Section:** Displays 'Workspace Status' as 'Draft in progress'. It includes a table for 'Year folder' (2026) and 'Saved setups' (1), and another table for 'Courses' (0) and 'Solutions' (0). A 'Save Setup In Database Year Folder' button is present.
- Saved Setups (Database):** Shows a list of saved setups: '2026' (1 setup) and 'Spring' (1 setup).
- Bottom Bar:** Includes 'Authenticated as orad.' and buttons for 'Validate Draft' and 'Generate Options'.

This screen shows the initial configuration step of the Exam Optimizer system. The user defines the project name, Moed window, exam date range, and minimum spacing constraints between exams. The side panel displays the current draft status, saved setups from the MongoDB database, and the setup progress. This stage prepares the basic scheduling rules before courses, fixed exams, and excluded dates are added.

8.2 Excluded Dates Interface – Blocking Unavailable Exam Periods

The screenshot shows the 'Excluded Dates' interface within the 'Exam Optimizer' application. On the left is a sidebar with a 'Setup' menu containing 'Initial Setup', 'Excluded Dates' (highlighted), 'Fixed Exams', 'Courses', and 'Schedule'. The main area has a header 'ACTIVE WORKSPACE' with a dropdown menu showing 'Spring Moed A'. Below this is the 'Excluded Dates' section, which includes a sub-header 'Block unavailable periods using an integrated date range picker.' and two input fields: 'Date Range' with a calendar icon and 'Reason' with a text input. An 'Add Range' button is to the right. A message at the bottom states 'No excluded ranges added yet.'

This screen allows the user to add unavailable date ranges that must be excluded from the exam schedule. The selected ranges, together with their reasons, are used as constraints during schedule generation so the solver will not place exams on blocked dates.

8.3 Fixed Exams Interface – Locking Exams to Required Dates

The screenshot shows the 'Fixed Exams' interface within the 'Exam Optimizer' application. The sidebar is identical to the previous screen, with 'Fixed Exams' highlighted in the 'Setup' menu. The main area has a header 'ACTIVE WORKSPACE' with a dropdown menu showing 'Spring Moed A'. Below this is the 'Fixed Exams' section, which includes a sub-header 'Lock required exams with modern date and combobox inputs.' and several input fields: 'Course Code', 'Course Name', 'Semester' (with a dropdown showing '1'), 'Exam Date' (with a calendar icon), 'Prerequisite Course Codes' (with a dropdown showing 'Select prerequisites'), and 'Department' (with a dropdown showing 'All departments'). There are two buttons: 'Add Fixed Exam' and 'Import Fixed Exams From Saved Setup'. A message at the bottom states 'No fixed exams yet.'

This screen allows the user to add exams with fixed dates that cannot be moved by the solver. Each fixed exam includes course details, semester, department, exam date, and prerequisite

information. These locked exams are included as hard constraints during schedule generation, ensuring that the generated timetable respects required exam dates.

8.4 Courses Interface – Managing Course Data for Scheduling

ACTIVE WORKSPACE

 Spring Moed A

Courses

High-density course data with search, sorting, pagination, and streamlined row actions.

Download TemplateImport Courses From Saved SetupImport Courses

Note: keep or delete the EXAMPLE row in each sheet. Rows that start with EXAMPLE are ignored during import.

Course Code

Course Name

Semester

Department

High Failure

1

All departments

No

Prerequisite Course Codes

Select prerequisites

Add Course

Search code, name, prerequisite...

Code ↕	Name ↕	Semester	Department	High Failure	Prerequisites
No results.					

Rows: 0

< Page 0 of 0 >

Authenticated as orad.

The Courses screen allows the user to manage the course list used by the exam scheduling algorithm. The user can manually add courses by entering the course code, course name, semester, department, prerequisite courses, and whether the course has a high failure rate. The interface also supports downloading an Excel template, importing courses from a saved setup, and importing course data from an external file. In addition, the table includes search, sorting, pagination, and row management features, making it easier to work with large course datasets before generating optimized exam schedules.

8.5 Schedule Interface - Viewing Generated Exam Timetable Options

The screenshot shows the 'Exam Optimizer' interface with a sidebar on the left containing 'Setup' and 'Schedule' (highlighted). The main area is titled 'Master Calendar' and 'Restored calendar view for solution-1'. It features a grid of exams organized by academic year (Year 1 to Year 4) and semester (Semester 1 to Semester 8). The rightmost column shows dates from 'Mon, June 15 - Day 1' to 'Sat, June 20 - Day 6'. Exams are represented by colored blocks with labels like 'PHY101 Physics 1 ALL', 'MAT202 Discrete Math IS', 'CS304 Software Engineering ALL', 'CS306 Distributed Systems ALL', 'MAT103 Calculus 2 ALL', and 'CS204 Operating Systems ALL'. A 'Conflicts' panel at the bottom right indicates 'No conflicts for this moed.' and includes buttons for 'Hide Conflicts', 'Optimize', and 'Save Solution To Database'. A status bar at the bottom left shows 'Loaded 'Spring' from database.'

The Schedule screen displays the generated exam timetable in a master calendar view. Exams are organized by academic year and semester, while the dates appear along the calendar axis. The user can switch between multiple generated solutions, filter the view by solution type, and inspect the schedule for a specific Moed window. Blocked dates, such as Saturdays, are visually marked and excluded from scheduling. The interface also includes a conflicts panel that summarizes high, medium, and active conflicts, helping the user verify that the selected timetable is valid before saving the final solution to the MongoDB database.

9. References

1. Burke, E. & Petrovic, S. (2002). Recent research directions in automated timetabling. *European Journal of Operational Research*.
2. Barták, R. et al. (2010). *Handbook of Constraint Programming*. Foundations of CSP-based scheduling.
3. Bonutti, A. et al. (2012). Curriculum-based course timetabling. *Annals of Operations Research*.
4. Aslan, Ö. & Kapanoglu, M. (2017). University exam scheduling via web-based decision support. *Computers & Education*.
5. Güler, M. et al. (2021). Examination timetabling with graph coloring and metaheuristics.
6. Pillay, N. & Banzhaf, W. (2010). An investigation into the use of hyper-heuristics for examination timetabling.
7. Elen, T. (2022). Metaheuristic approaches to examination timetabling.
8. Google OR-Tools Documentation. <https://developers.google.com/optimization>
9. FastAPI Documentation. <https://fastapi.tiangolo.com>
10. *Automated System for University Exam Scheduling – Phase A Report* (Project Code: 26-1-D-12). Braude College of Engineering. <https://github.com/KarynA19/Exam-Optimizer>

10. User Guide

10.1 Introduction

This guide explains how to use Exam Optimizer, a web-based tool for building final exam timetables for the Software/Information Systems Engineering Department. It is intended for the scheduling coordinator, who defines exam windows, enters courses, generates schedule options, compares them, and applies small adjustments.

10.2 Getting Started

Accessing the system

1. Open the Exam Optimizer URL in a modern web browser (Chrome, Firefox, Safari, or Edge). <https://exam-optimizer-4sf.pages.dev/setup>
2. The application opens on the Setup page.
3. Your work is saved automatically in the browser but to be able to save solutions per year/semester, you would need to login and save it to the database folder (folder on the right)
4. Login using the following credentials:
Id: orad
Password: braude2026

Main navigation

The left sidebar has two areas:

- Setup: configure the exam period, blocked dates, fixed exams, and courses.
- Schedule: review generated timetables, compare options, and adjust exam dates.

Under Setup, four sub-steps appear after the initial setup is saved:

1. Initial Setup
2. Excluded Dates
3. Fixed Exams
4. Courses

10.3 Nominal workflow overview

The typical end-to-end process:

1. Save Initial Setup (Moed windows and gap rules).
2. Add Excluded Dates (holidays and blocked periods).
3. Add Fixed Exams (pre-confirmed dates), if any - or skip and import them in Courses.
4. Add or import the Courses list.
5. Click Generate options to produce up to five timetable candidates.

6. Open Schedule, compare solutions, and refine dates with manual moves if needed.

10.4 Setup – Initial Configuration

Step 1: Initial Setup

1. In the sidebar, open **Setup** → **Initial Setup**.
2. Enter a **Project Name** such as Spring 2026.
3. Set the **Moed Count** according to the number of exam periods required, for example Moed A, Moed B, and Moed C.
4. For each Moed window, define the following fields:
 - **Start Date** and **End Date**
 - **Same Semester Gap** – the minimum number of days between exams from the same semester
 - **Prerequisite Gap** – the minimum number of days between courses connected by prerequisite relations
 - **High-Failure Gap** – the minimum spacing required around high-failure courses and related nearby semesters
5. Click **Save Setup In Database Year Folder** to save the initial configuration.

After the initial setup is saved, the remaining setup sections become available: **Excluded Dates**, **Fixed Exams**, and **Courses**. Saved setups are grouped by year in the right-side panel and can be reused in future semesters. To reuse a saved setup, open the **Saved Setups** panel, select the relevant year folder, and click the setup name to load it into the workspace.

Step 2: Excluded Dates

1. Go to **Setup** → **Excluded Dates**.
2. Select the unavailable date range using the date picker.
3. Enter a short **Reason**, such as Holiday, Ceremony, or Maintenance.
4. Click **Add Range**.
5. Repeat this process for every blocked period.

These dates are removed from the solver's allowed date domain, so generated schedules will not place exams during blocked periods.

Step 3: Fixed Exams

This step can be skipped if there are no pre-agreed exam dates.

1. Go to **Setup** → **Fixed Exams**.
2. Enter the **Course Code**, **Course Name**, **Semester**, and **Exam Date**.
3. Optionally select **Prerequisite Course Codes** and **Department**.
4. Click **Add Fixed Exam**.
5. Repeat this process for every exam that must remain on a specific date.

Fixed exams are treated as hard constraints. The solver must preserve their selected dates when generating the timetable.

Step 4: Courses

Courses can be added manually, imported from an Excel file, or loaded from a previously saved setup.

Option A: Import from Excel

1. Go to **Setup** → **Courses**.
2. Click **Download Template** to download the Excel template.
3. Fill in the required course data:
 - Course ID
 - Course Name
 - Semester
 - Is High Failure
 - Prerequisites
 - Department
4. The template also includes a separate tab for fixed exams, making it easier to prepare both course data and locked exam dates in one file.
5. The first row in each sheet is an example row. Rows marked as example rows are ignored during import.
6. Click **Import Courses**, select the completed .xlsx file, and wait for the import process to finish.

Importing a new file replaces the current course list in the workspace.

Option B: Add Courses Manually

1. Fill in the **Course Code**, **Course Name**, and **Semester**.
2. Optionally select **Prerequisite Course Codes**, **Department**, and **High Failure** status.
3. Click **Add Course**.
4. Repeat until all courses have been added.

Option C: Import Courses from a Previous Setup

1. Go to **Setup** → **Courses**.
2. Click **Import Courses From Saved Setup**.
3. Select an existing saved setup from the database.
4. The system loads the course list from the selected setup into the current workspace.
5. Review the imported courses and update them if needed before generating schedules.

Step 5: Generate Schedule Options

1. At the bottom of the Setup page, click **Validate Draft** to check whether the current configuration is valid. This step is optional but recommended.
2. Click **Generate Options**.
3. Wait while the system runs the CP-SAT solver and generates up to five timetable candidates.
4. When the process finishes successfully, the system opens the **Schedule** workspace.
5. In the Schedule workspace, the user can review the generated timetables, switch between solutions, inspect conflicts, and save the selected solution to the database.

10.5 Schedule Workspace – Reviewing Timetables

After the schedule generation process is completed, the user is redirected to the **Schedule** workspace. This page is used to review the generated timetable options, compare alternative solutions, inspect conflicts, and make small manual adjustments when needed.

Using the Master Calendar

1. Open the **Calendar** tab.
2. Select the relevant Moed window, such as **Moed A** or **Moed B**, if more than one window was defined.
3. Optionally filter the displayed exams by department: **All**, **SW**, or **IS**.
4. Choose between the different solutions.
5. Review the exams in the calendar grid, where courses are grouped by academic year, semester, and assigned exam date.

Blocked dates, such as Saturdays or excluded periods, are visually marked in the calendar and cannot contain scheduled exams.

The **Show Changes** option highlights exams that were manually moved compared to the originally generated solution. The **Reset to Optimal** option restores the selected solution to its original solver-generated state.

The user can also open the conflicts panel to review any detected issues before deciding whether to save the selected timetable to the database.

10.6 Manual Exam Adjustments

Manual exam adjustments allow the user to move a single exam to a different date without generating the entire timetable again. This is useful when the coordinator wants to make a small correction while still keeping the schedule valid.

1. Open the Calendar view in the Schedule workspace.

2. Click the exam chip that you want to move.
3. The system checks the possible target dates in the selected Moed window and displays a move preview on the calendar:

Green: the move is valid, and the solution score stays the same or improves.

Yellow: the move is valid, but the solution score decreases.

Red: the move is not valid (but will still be allowed, as requested) because it violates one or more scheduling rules.

4. Click and drag to a date.
5. Open the Conflicts panel to review the selected date and any rule details related to the move.
6. After applying the move, the timetable is updated and saved automatically in the local browser draft.
7. Any change to the solution can be saved to the database using the designated button.

The user can also make a manual adjustment from the Solution Details section by changing the exam date in the table and applying the move from there.

Manual moves are validated by the backend before they are applied. This ensures that the user can refine the timetable without accidentally creating an invalid schedule.

10.7 Continuing your work

- To start a new semester, save a new Initial Setup or load an existing one from Saved Setups, then update excluded dates, courses, and generate new options.
- To return to configuration after solving, click Setup in the sidebar.

11. Maintenance Guide

11.1 Technology Stack

Technology	Purpose	Version
React	UI library (single-page application)	18.3.1
TypeScript	Type-safe frontend code	5.8.3
Vite	Frontend build tool and dev server	5.4.19

Cloudflare Pages	Frontend hosting and deployment	Latest
Browser localStorage	Client-side project persistence	Web API
FastAPI	Backend REST API framework	0.115.12
Python	Backend runtime	3.11
Uvicorn	ASGI server for the API	0.34.2
Pydantic	Data models and request validation	2.11.4
Google OR-Tools (CP-SAT)	Constraint programming solver	9.12.454 4
openpyxl	Excel (.xlsx) course import and template export	3.1.5
python-multipart	File upload handling	0.20.0
pytest	Automated backend unit tests	8.3.5
Docker	Backend containerization and deployment	Latest

11.2 Development Environment Setup

11.2.1 Prerequisites

Before running the project locally, install the following tools:

Node.js: Version 18 or above, required for the React frontend.

npm: Used for installing and running frontend dependencies.

Python: Version 3.11, matching the backend Docker image.

pip: Used for installing backend Python dependencies.

Git: Used for cloning the project repository.

MongoDB: Required for saving and loading setups and solutions from the database. MongoDB can be installed locally or accessed through a remote MongoDB connection string.

Docker: Optional, used for backend containerization and deployment.

11.2.2 Installing Dependencies

Clone the repository and install dependencies for both the backend and frontend.

Clone the repository:

```
git clone https://github.com/KarynA19/Exam-Optimizer.git  
cd Exam-Optimizer
```

Backend setup:

```
cd backend  
python -m venv .venv
```

Activate the virtual environment.

For Windows:

```
.venv\Scripts\activate
```

For macOS / Linux:

```
source .venv/bin/activate
```

Install backend dependencies:

```
pip install -r requirements.txt
```

Frontend setup:

```
cd ../frontend  
npm install
```

11.2.3 Environment Variables

The backend loads environment variables from an optional .env file located inside the backend/ folder. Environment variables may also be set directly in the shell or deployment platform.

Backend .env example:

```
# Comma-separated list of allowed frontend origins for CORS  
ALLOWED_ORIGINS=http://localhost:5173,https://your-cloudflare-pages-url.pages.dev
```

Port used by Uvicorn in Docker or production deployment
PORT=8080

MongoDB connection string
MONGODB_URI=mongodb://localhost:27017

MongoDB database name
MONGODB_DATABASE=exam_optimizer

Single-user login credentials
SINGLE_USER_ID=orad
SINGLE_USER_PASSWORD=braude2026

JWT authentication settings
JWT_SECRET=change-this-secret-in-production
JWT_EXPIRATION_MINUTES=10080

For local development, MongoDB must be running if the user wants to save setups or solutions to the database. If no custom MongoDB variables are provided, the backend uses:

MONGODB_URI=mongodb://localhost:27017
MONGODB_DATABASE=exam_optimizer

The saved setup data is stored in the MongoDB collection:

saved_setups

Frontend `.env` example:

Backend API base URL for production builds
VITE_API_BASE=<https://your-backend-api-url/api>

In local development, if VITE_API_BASE is not set, the frontend automatically uses:

<http://localhost:8000/api>

11.2.4 Running the Development Server

Run the backend from the backend/ folder with the virtual environment active:

uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

The backend API is available at:

<http://localhost:8000>

Health check endpoint:

GET <http://localhost:8000/health>

Main API routes are available under:

/api

Important backend endpoints include:

POST /api/auth/login

POST /api/projects/validate

POST /api/projects/solve

POST /api/projects/manual-move

POST /api/projects/explain-move

POST /api/projects/import-courses

GET /api/projects/import-courses/template

GET /api/saved-setups

POST /api/saved-setups

GET /api/saved-setups/{setup_id}

DELETE /api/saved-setups/{setup_id}

PUT /api/saved-setups/{setup_id}/solutions

GET /api/saved-setups/{setup_id}/courses

GET /api/saved-setups/{setup_id}/fixed-exams

Run the frontend from the frontend/ folder:

npm run dev

This starts the frontend development server at:

<http://localhost:5173>

To create a production build:

npm run build

To preview the production build locally:

npm run preview

Run backend tests from the backend/ folder:

pytest

11.2.5 Project Structure

Exam-Optimizer project structure:

Exam-Optimizer/

- └─ backend/
 - └─ app/
 - └─ api/
 - └─ routes/
 - └─ auth.py # Login API endpoint
 - └─ projects.py # Project validation, solving, manual move, and import endpoints
 - └─ saved_setups.py # MongoDB saved setup and solution endpoints
 - └─ models/
 - └─ schedule.py # Pydantic data models
 - └─ services/
 - └─ auth.py # JWT authentication and single-user login logic
 - └─ course_import.py # Excel import and template export
 - └─ saved_setup_store.py # MongoDB persistence for saved setups and solutions
 - └─ solver.py # CP-SAT scheduling engine
 - └─ validation.py # Rule checking, scoring, and manual move validation
 - └─ main.py # FastAPI app entry point and router registration
 - └─ tests/
 - └─ test_solver.py # Solver tests
 - └─ test_course_import.py # Excel import tests
 - └─ Dockerfile # Backend Docker configuration
 - └─ requirements.txt # Backend Python dependencies
 - └─ frontend/
 - └─ src/
 - └─ api.ts # Backend communication and authentication token handling
 - └─ App.tsx # Main application shell
 - └─ main.tsx # React entry point
 - └─ types.ts # TypeScript types matching backend models
 - └─ storage/
 - └─ localProject.ts # Browser localStorage load/save logic
 - └─ components/
 - └─ common/ # Shared UI components
 - └─ forms/ # Setup forms for courses, exclusions, and fixed exams
 - └─ ui/ # Reusable UI components
 - └─ workspace/ # Master calendar, solution cards, and conflict drawer
 - └─ lib/
 - └─ utils.ts # Shared frontend utility functions
 - └─ utils/
 - └─ calendarUtils.ts # Calendar layout helpers
 - └─ dateHelpers.ts # Date formatting and date logic
 - └─ departmentUtils.ts # Department-related helper functions
 - └─ examKeys.ts # Exam key generation helpers
 - └─ workspaceUtils.ts # Workspace helper logic

```
| |— index.html
| |— package.json
| |— package-lock.json
| |— tsconfig.json
| |— vite.config.ts
|
|— README.md
|— .gitignore
```

11.3 Backend Architecture

11.3.1 API Routes Structure

The backend is implemented with FastAPI and exposes REST API routes under the /api prefix. The backend is responsible for validation, schedule generation, manual move checking, Excel import, authentication, and persistence of saved setups and solutions in MongoDB.

The FastAPI application is created in app/main.py. It registers three main route modules:

auth.py – handles login and JWT token generation.

projects.py – handles validation, solving, manual moves, move explanations, and Excel import.

saved_setups.py – handles saving, loading, deleting, and updating saved setups and solutions in MongoDB.

The backend also exposes a health endpoint:

GET /health

This endpoint is useful for checking whether the deployed backend service is running correctly.

11.3.2 Core API Endpoints

Endpoint	Method	Purpose
/health	GET	Service health check
/api/auth/login	POST	Authenticate the user and return a JWT token
/api/projects/validate	POST	Validate the project setup, including courses, Moed windows, fixed exams, and excluded dates
/api/projects/solve	POST	Run the CP-SAT solver and return up to five timetable options
/api/projects/manual-move	POST	Validate and score a proposed exam date change
/api/projects/explain-move	POST	Evaluate a possible move for a specific date; used for calendar preview coloring
/api/projects/import-courses	POST	Import courses and fixed exams from an .xlsx file
/api/projects/import-courses/template	GET	Download the Excel course import template
/api/saved-setups	GET	List saved setups from MongoDB for the authenticated user
/api/saved-setups	POST	Save a new setup or update an existing setup in MongoDB
/api/saved-setups/{setup_id}	GET	Load a full saved setup from MongoDB
/api/saved-setups/{setup_id}	DELETE	Delete a saved setup from MongoDB

/api/saved-setups/{setup_id}/solutions	PUT	Save or update generated solutions for an existing setup
/api/saved-setups/{setup_id}/courses	GET	Load only the course list from a saved setup
/api/saved-setups/{setup_id}/fixed-exams	GET	Load only the fixed exams from a saved setup

11.3.3 Request Bodies

The main backend request bodies are:

Login request:

```
{ user_id: string, password: string }
```

Validate request:

Full ScheduleProject object

Solve request:

```
{ project: ScheduleProject, max_solutions: number }
```

Manual move / explain move request:

```
{ project: ScheduleProject, solution_id: string, course_code: string, moed_number: number, new_date: string }
```

Import courses request:

multipart/form-data with a file field containing an .xlsx file

Save setup request:

```
{ project: ScheduleProject, setup_id?: string, saved_solution_id?: string }
```

Update setup solutions request:

```
{ solutions: ScheduleSolution[], saved_solution_id?: string }
```

11.3.4 Service Layer

Module	Responsibility
validation.py	Builds allowed dates, checks blocked dates, validates gap rules, Friday limits, fixed exams, scoring, and manual move previews
solver.py	Builds and solves the CP-SAT model, generates multiple timetable options, and combines Moed window results
course_import.py	Parses Excel files, validates imported course data, imports fixed exams, and exports the Excel template
saved_setup_store.py	Connects to MongoDB and stores saved setups, selected solutions, course lists, and fixed exams
auth.py	Handles single-user login, JWT token creation, and authentication checks for protected saved-setup routes
schedule.py	Defines Pydantic schemas for all request and response objects

11.4 Frontend Architecture

11.4.1 Vite + React SPA

Exam Optimizer uses a single-page application built with Vite, React, and TypeScript.

Entry point: *main.tsx renders App.tsx into #root*

Routing: *Client-side routing is handled through window.location using /setup and /schedule*

Styling: *Global styling is defined in styles.css, with reusable UI components under components/ui*

Backend communication: *api.ts contains all fetch calls to the FastAPI backend, including authentication, validation, solving, importing, and saved setup operations*

Authentication state: *The frontend stores the JWT token and user id in browser localStorage*

11.4.2 Application Routes

Route	Purpose
/setup	Main project configuration page
/setup#project	Initial setup: project name, Moed windows, and gap rules
/setup#excluded	Excluded date ranges
/setup#fixed	Fixed exams
/setup#courses	Course list, Excel import, and importing courses from saved setups
/schedule	Review generated solutions, inspect the calendar, compare options, and perform manual moves

11.4.3 Component Organization

Folder / File	Contents
components/forms/	CourseForm, ExcludedRangeForm, FixedExamForm
components/workspace/	MasterCalendar, ConflictDrawer, SolutionCard
components/common/	SectionTitle, IssueList, Metric, EditableSimpleList
components/ui/	Reusable UI components such as buttons, cards, calendar, dropdowns, tabs, forms, sidebar, and data table
storage/	localProject.ts, responsible for loading and saving the browser draft and local setup library
utils/	Calendar helpers, date helpers, department helpers, exam key helpers, and workspace utilities

lib/	Shared frontend utility functions
api.ts	All fetch calls to the backend, including auth token handling and MongoDB saved setup API calls
App.tsx	Main application shell and central state management

11.4.4 State Management

The frontend state is mainly managed inside App.tsx using React state hooks.

Project state:

The current project draft is stored in React state and automatically saved to browser localStorage after changes.

Local draft persistence:

The file localProject.ts saves and loads the current project from browser localStorage using the key:

exam-optimizer:project

Local setup library:

The frontend also supports a local setup library stored in localStorage using the key:

exam-optimizer:setup-library

Remote saved setups:

In the updated version, saved setups and solutions can also be stored in MongoDB through the /api/saved-setups endpoints. These routes require login and use a JWT token.

Authentication state:

The frontend stores the login token and user id in localStorage using:

exam_optimizer_auth_token

exam_optimizer_auth_user_id

Server state:

The backend does not keep temporary project state between validate, solve, or manual-move requests. Instead, the frontend sends the full project object in each request. Persistent setups and selected solutions are stored only when the user explicitly saves them to MongoDB.

Preview state:

During manual move preview, the frontend sends explain-move requests for possible target dates and stores the returned preview responses for calendar coloring.

11.5 Data Persistence Architecture

11.5.1 Data Flow

Exam Optimizer uses two persistence layers:

1. Browser localStorage for temporary local drafts.
2. MongoDB for saved setups and saved solutions.

The local workflow is:

The coordinator edits the project in the browser.

The frontend updates React state.

The current draft is automatically saved to browser localStorage.

When the user validates, solves, or previews a manual move, the full project is sent to the backend in the request body.

The backend computes the result and returns it to the frontend.

The frontend updates the local project state and saves the updated draft back to localStorage.

The MongoDB workflow is:

The user logs in through /api/auth/login.

The backend returns a JWT token.

The frontend stores the token in localStorage.

When the user saves a setup, the frontend sends the project to /api/saved-setups.

The backend stores the setup in MongoDB under the saved_setups collection.

Generated solutions can later be saved to the same setup through /api/saved-setups/{setup_id}/solutions.

Courses and fixed exams can also be loaded from a saved setup through dedicated endpoints.

This means that browser drafts are convenient for continuing work on the same machine, while MongoDB provides central persistence for saved setups and selected generated solutions.

11.5.2 MongoDB Persistence

MongoDB is used for persistent storage of saved setups and solutions.

The main collection is:

saved_setups

Each saved setup is associated with an authenticated owner id. The saved setup document stores metadata such as setup id, project name, year, save time, update time, and selected saved solution id. It also stores the full project data, including Moed windows, courses, fixed exams, excluded ranges, and generated solutions when they are saved.

Maintenance implication:

If the browser draft is cleared, the local unsaved draft may be lost. However, setups and solutions that were saved to MongoDB can be loaded again after login.

11.6 Development Workflows

11.6.1 Git Workflow

Repository: <https://github.com/KarynA19/Exam-Optimizer.git>

Branches: Create feature branches from main.

Commits: Use descriptive commit messages focused on the purpose of the change.

Pull requests: Recommended before merging changes into main.

Code review: At least one team member should review important changes before merge.

11.6.2 Backend Workflow

Typical backend development flow:

```
cd backend
source .venv/bin/activate
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Run tests before merging backend changes:

```
pytest
```

Important backend areas to test after changes:

Solver behavior in solver.py.

Validation behavior in validation.py.

Excel import and template generation in course_import.py.

MongoDB save/load behavior in saved_setup_store.py.

Authentication behavior in auth.py.

11.6.3 Frontend Workflow

Typical frontend development flow:

```
cd frontend  
npm run dev
```

Before deployment, create a production build:

```
npm run build
```

Preview production build locally:

```
npm run preview
```

Important frontend areas to test after changes:

Setup flow.

Course import.

Saved setup loading from MongoDB.

Schedule generation.

Manual move preview.

Saving selected solutions.

11.7 Debugging and Troubleshooting

11.7.1 HTTP Status Codes Reference

Code	Meaning	Common Cause	Solution
200	OK	Request succeeded	No action needed
401	Unauthorized	Missing, invalid, or expired JWT token	Log in again and retry the saved setup action
422	Unprocessable Entity	Validation failure, invalid Excel import, or infeasible solve	Check the response details and fix the reported validation issues
500	Internal Server Error	Backend crash, solver error, database error, or unexpected import error	Check Uvicorn or Docker logs and reproduce with tests

11.7.2 Common Issues and Solutions

Problem	Cause	Solution
Frontend cannot reach API	Wrong or missing VITE_API_BASE	Set VITE_API_BASE to the backend URL ending in /api, then rebuild and redeploy
CORS error in browser	Frontend origin is not listed in ALLOWED_ORIGINS	Add the Cloudflare Pages URL to ALLOWED_ORIGINS on the backend
"Missing VITE_API_BASE for production build."	Production build was created without the required environment variable	Set VITE_API_BASE before running npm run build
Login fails	Wrong user id or password, or backend environment variables do not match	Check SINGLE_USER_ID and SINGLE_USER_PASSWORD in the backend environment
Session expired	JWT token is missing, invalid, or expired	Log in again
Saved setups do not load	MongoDB is not running or MONGODB_URI is incorrect	Start MongoDB locally or fix the remote MongoDB connection string
Save setup fails with 401	User is not authenticated	Log in before saving or loading saved setups
Solve returns 0 solutions	Infeasible constraints, such as narrow Moed window, tight gaps, or conflicting fixed exams	Widen the Moed window, relax gap days, reduce fixed exams, or read the no_feasible_schedule message
Draft lost after browser change	Local draft is stored per browser using localStorage	Use MongoDB saved setups for persistent cross-browser storage

Course import fails	Invalid .xlsx format or missing columns	Download the template from the UI and verify the columns: Course ID, Course Name, Semester, Is High Failure, Prerequisites, Department
OR-Tools import error on Windows	Environment or path issue	Use Python 3.11, reinstall requirements, and avoid special characters in the project path
Explain-move feels slow	The frontend sends preview requests for multiple possible dates	Expected behavior; the requests are used to color the calendar preview

11.8 Exam Optimizer – Specific Features

11.8.1 Scheduling Policies Encoded in Code

Policy	Type	Location
Same-semester gap per Moed window	Hard constraint	validation.py, solver.py
Prerequisite gap for linked back-to-back semester courses	Hard constraint	validation.py, solver.py
High-failure course gap from courses in the two preceding semesters	Hard constraint	validation.py, solver.py
Adjacent-semester spacing	Hard constraint / scoring preference	validation.py, solver.py
One Friday exam per department per Moed window	Hard constraint	validation.py, solver.py
No exams on Saturdays or excluded dates	Hard constraint	validation.py, solver.py

Fixed exams on assigned dates	Hard constraint	solver.py, validation.py
Even distribution across the Moed window	Soft scoring objective	validation.py, solver.py
Friday penalty	Soft scoring objective	validation.py, solver.py
Same-day and same-parity penalties	Soft scoring objective	validation.py, solver.py
Courses without a department	Count as both SW and IS	course_departments() in validation.py
Manual move validation	Backend validation before applying move	validation.py, projects.py
Saved setups and solutions	Persistent database storage	saved_setup_store.py, saved_setups.py, MongoDB
Login and protected database actions	Authentication	auth.py, api.ts

11.9 Future Maintenance

Monitoring

Monitor the backend /health endpoint uptime.

Track solve response times. The target is to keep solving under 60 seconds for a typical department-sized project.

Track the frequency of `no_feasible_schedule` responses to detect whether default gap settings are too strict.

Monitor MongoDB availability, because saved setups and saved solutions depend on the database connection.

Check authentication errors to identify expired sessions or incorrect deployment environment variables.

Optimization

Reduce explain-move API calls by adding a batch preview endpoint that evaluates all target dates in one request.

Optimize solving algorithm if using a weak server

Cache allowed-date lists per project hash on the backend for repeated manual-move preview calls.

Add MongoDB indexes if the number of saved setups grows significantly.

Improve frontend loading states for save/load operations.

Security

Monthly: run *npm audit* for frontend dependencies and review Python dependency updates.

Per release: verify that ALLOWED_ORIGINS matches the current Cloudflare Pages URL.

Per deployment: replace the default JWT_SECRET with a secure production value.

Avoid storing real passwords directly in source code.

Consider improving the current single-user login into a more secure multi-user authentication model.

Planned Extensions

Lecturer availability constraints.

Full automatic re-shuffle of unlocked courses.

Extension to additional university departments.

Improved login security and role-based access.

Batch manual-move preview endpoint.

More detailed MongoDB backup and restore workflow.

Better reporting/export of final timetables.